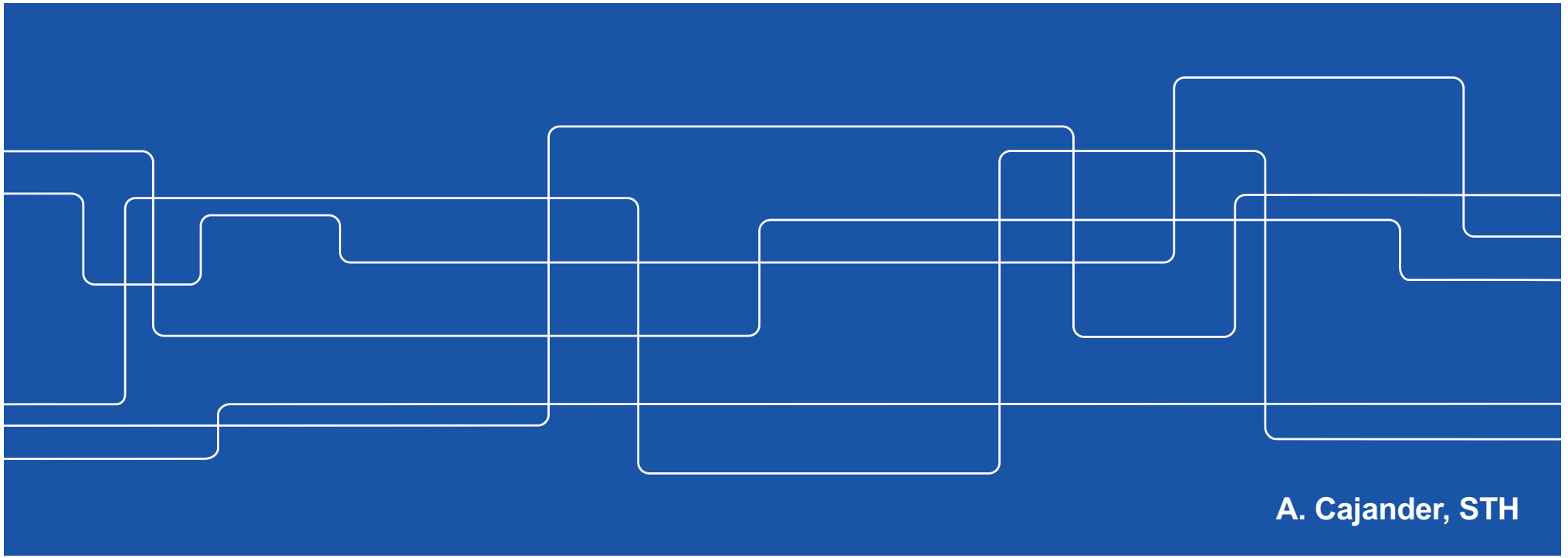
HE1025 Operativsystem

L3: Concurrency



A. Cajander, STH

Kursen…

F1: V#1 Processorn.	Kap (1) 2..4 & 6
Ö1: V#2 c Pekare…	Kap 14 & c-boken
F2: V#3 Schemaläggning	Kap 7..9,(10-11), RTOS
L1: V#4 FreeRTOS	Lab #1 Handledning
F3: V#5 Segmentation	Kap 12..13, 15..17
Ö2: V#6	
F4: V#7 Paging	Kap 18-22, (23-24)
L2: V#8 Paging	Lab #2 Handledning
F5: C#1 Locks & CV	Kap (25) 26..27 & 28..29.1
Ö3: C#2 LINUX	Kap 5 (39)
F6: C#3 Semaphores	Kap 30..33, (34)
L3: C#4 Farmacy	**Lab #3 Handledning**
F7: P#1 Storage	Kap (35,) 36, 40, 42 (37..39, 41 & 43..46)
Ö4: P#2 File/DMA	.ppt
F8: P#3 Communication	Kap (47, 39) & 48, (49..51)
L4: P#4 Client/Server	Lab #4 Handledning

Virtualization
Concurrency
Persistenc

TEN1: 4+4p/4:e-del, minst 4p per 4:e-del för E!
LAB1: Närvaro L1..4

Agenda

x+0.15 FreeRTOS
x+0.30 Grupparbete Tasks (Lab #1)

x+0.45 Apotek & Queue
Grupparbete Poisson process

Queue
x+1.15 Grupparbete Thread safe LCD

x+1.25 Lås & Semaforer
x+1.30 Grupparbete Simulering

x+1.45 Sammanfattning

FreeRTOS

Har utvecklats av ledande aktörer allt sedan millennieskiftet…
…överlägset störts på marknaden, laddas ner var annan min!

Allmänt ger ett RTOS:

- Abstraherar bort timing information
- Underlättar utveckling, testning & underhåll
- Prestanda & Power management (idle/int)

Specifikt FreeRTOS därutöver:

- Beprövad, gratis, minimal kodbas (3 filer!)
- Omfattande developer community

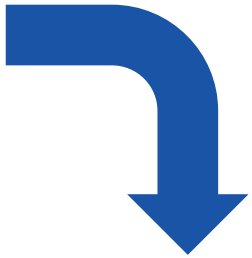
Features

- Pre-emptive or co-operative operation
- Very flexible task priority assignment
- Flexible, fast and light weight task notification mechanism
- Queues
- Binary semaphores
- Counting semaphores
- Mutexes
- Recursive Mutexes
- Software timers
- Event groups
- Tick hook functions
- Idle hook functions
- Stack overflow checking
- Trace recording
- Task run-time statistics gathering

FreeRTOS

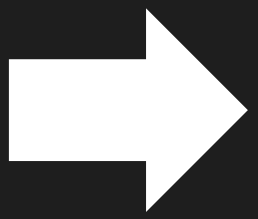
```
while (1) {
        idle++;                 // Manage Performance Monitoring!
        LCD_WR_Queue();         // Manage LCD com queue!
        U0_TX_Queue();          // Manage U(S)ART TX Queue!
        ADC0_Management();      // Manage ADC0 conversion!
        DAC0_Management();      // Manage DAC0 conversion!
        *
        *
        *
        LED_Panel();            // Manage 8*8 LED Panel!
}
```

(Super Loop)

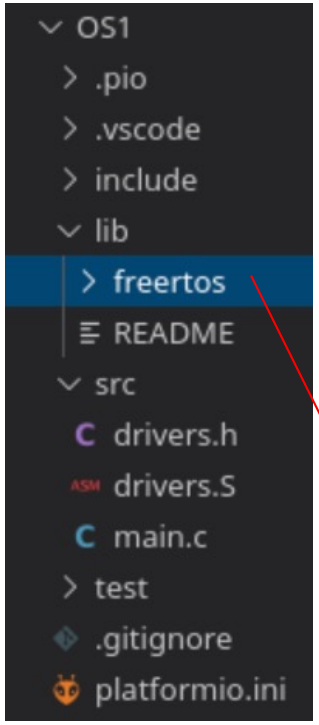


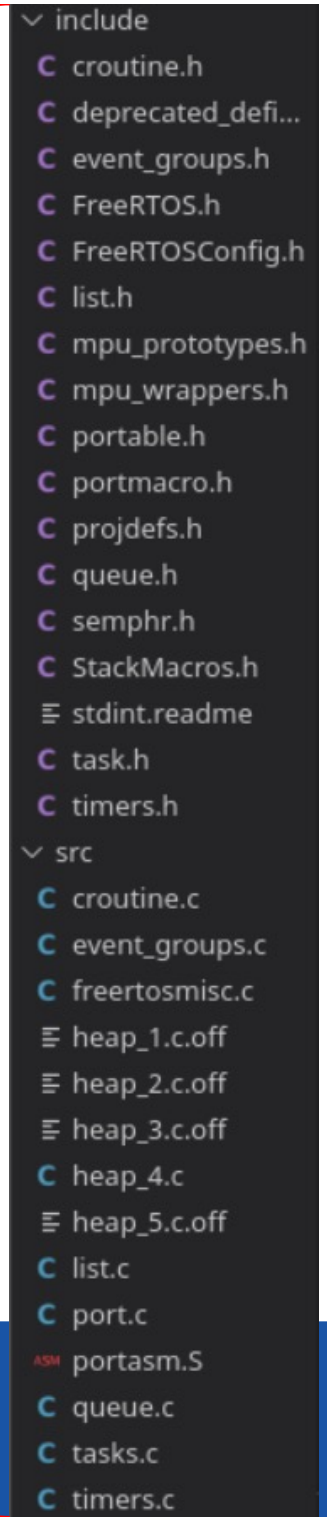
```
int main(void)
{
    xTaskCreate(perfMonitor, "task1", 512,(void*)NULL, 2, NULL);
    xTaskCreate(LCD_WR_Queue, "task2", 512,(void*)NULL, 2, NULL);
    xTaskCreate(U0_TX_Queue, "task3", 512,(void*)NULL, 2, NULL);
    *
    *
    *
    xTaskCreate(LED_Panel, "task12", 512,(void*)NULL, 2, NULL);

    vTaskStartScheduler();
    // Should not reach here...

    while(1);  // ...to be sure.
}
```

Varje “uppgift” blir ett ”eget” program med en egen Super loop!



2ms Time slice

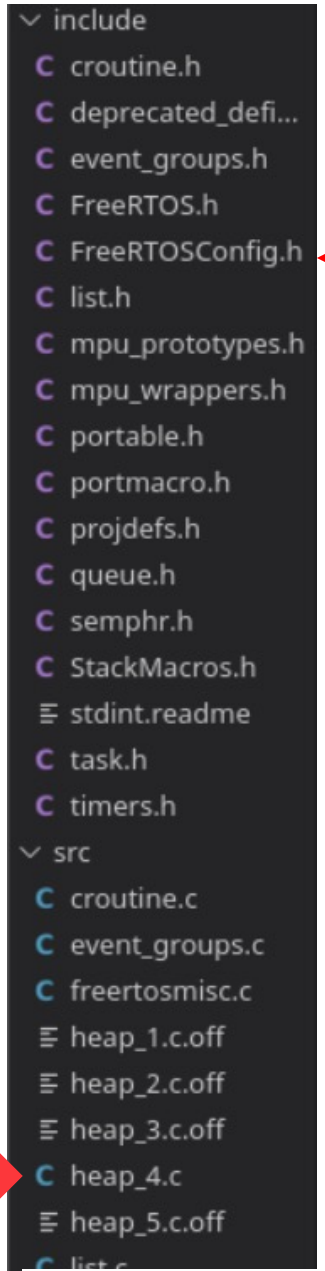




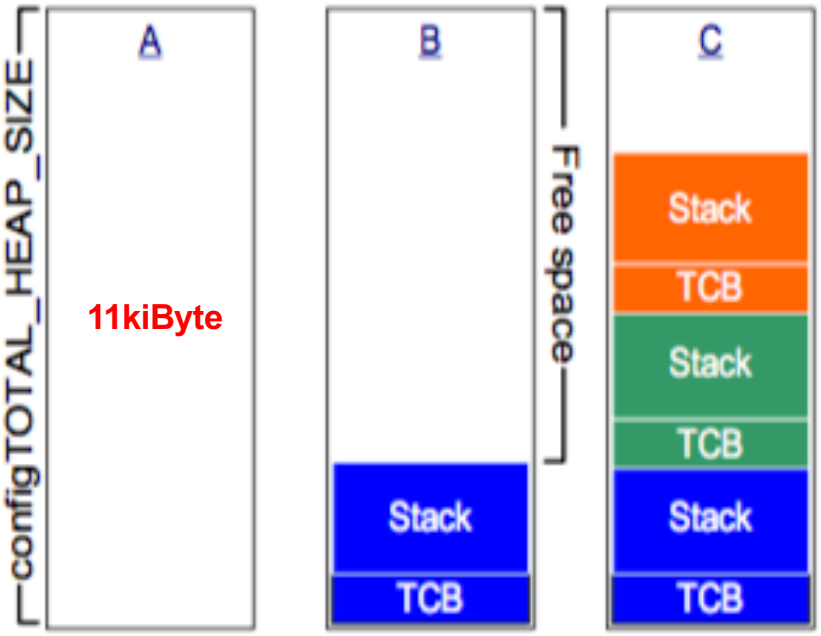
Alla filer till vänster, utom drivers & main, tillhör FreeRTOS och utgör os:et. Den koden ska inte röras, men måste finnas med i projektet, all koppling sker via ett fåtal väl specificerade api-anrop! I Canvas finns foldern ”freertos” som ni bara laddar ner och placerar i projektets folder ”lib”, så kommer det att fungera!

```c
#include "FreeRTOS.h" /* Must come first. */
#include "task.h"     /* RTOS task related API prototypes. */

#include "gd32vf103.h"// Main MCU API header
#include "drivers.h"  // LED Panel & Keyboard driver API

int main(void)
{
    // Perform any driver setup necessary...

    // Create (initially) needed application tasks...

    // Start the Scheduler!
    vTaskStartScheduler();
    // Should not reach here...

    while(1); // ...to be sure!
}
```

Memory Allocation...



configTOTAL_HEAP_SIZE



- heap_1 - the very simplest, does not permit memory to be freed.
- heap_2 - permits memory to be freed, but does not coalescence adjacent free blocks.
- heap_3 - simply wraps the standard malloc() and free() for thread safety.
- heap_4 - coalescences adjacent free blocks to avoid fragmentation. Includes absolute address placement option.
- heap_5 - as per heap_4, with the ability to span the heap across multiple non-adjacent memory areas.

First-fit

Task Management...

Ett TCB är 112 Byte

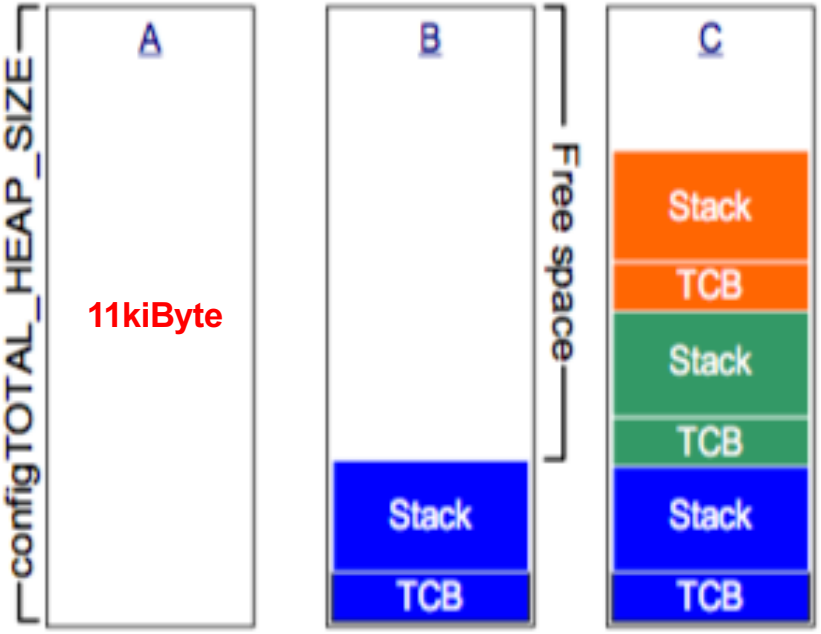
Stack 2048 Byte*

M.a.o 2160 Byte/task!

Teoretiskt

11*1024/2160=5 task!

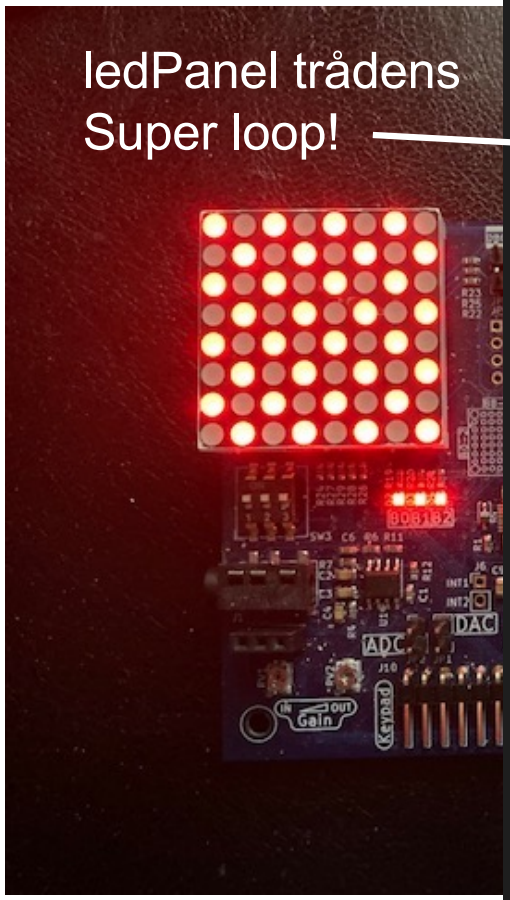
(*Ska räcka till “alla” funktioners stack frames för den “längsta” anrops-kedjan i programmet + context switch data!)



void aTaskFunction(void *pvParameters); (never end)

```
BaseType_t xTaskCreate(
                TaskFunction_t pvTaskCode,          //code
                const char * const pcName,          //name
                uint16_t usStackDepth,              //stack
                void *pvParameters,                 //param
                UBaseType_t uxPriority,             //prio
                TaskHandle_t *pxCreatedTask );      //Id
```

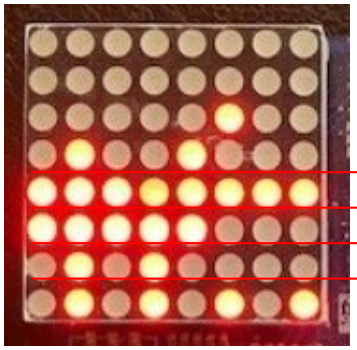
ledPanel trådens Super loop!



```c
#include "FreeRTOS.h" /* Must come first. */
#include "task.h"     /* RTOS task related API prototypes. */

#include "gd32vf103.h"// Main MCU API header
#include "drivers.h"  // LED Panel & Keyboard driver API

void ledPanel(void *pvParameters){      // The Led Panel Task:
    while (1) {                         // Led Panel Super Loop...
        l88row(colset());               // ...show next row...
        vTaskDelay(1);                  // ...wait 2ms and repeat!
    }                                   // (Must not terminate)
}

int main(void)
{
    // Perform any driver setup necessary...
    colinit();                          // Initialize column toolbox
    l88init();                          // Initialize 8*8 led toolbox

    // Create (initially) needed application tasks...
    xTaskCreate(ledPanel, "8*8LED", 512,(void*)NULL, 2, NULL);

    // Start the Scheduler!
    vTaskStartScheduler();
    // Should not reach here...

    while(1); // ...to be sure!
}
```

Back in business…

xPortGetFreeHeapSize()



0x00000448

“ledTask1”
“ledTask2”
“ledTask3”

“Räknar upp” värdet i raden med 1 varje sekund, var annan sekund och var 3:e sekund

```
8   void ledPanel(void *pvParameters){      // The Led Panel Task:
9       [redacted]
10      while (1) {                         // Led Panel Super Loop...
11          l88row(colset());               // ...show next row...
12          vTaskDelay(1);                  // ...wait 2ms and repeat!
13
14          l88mem(0,
15          l88mem(1,
16          l88mem(2,
17          l88mem(3,
18      }                                   // (Must not terminate)
19  }
20
21  void ledTask(void *pvParameters){       // The Led Panel Task:
22                                          ters;
23                                          // Led Panel Super Loop...
24
25                                          // ...wait 2+id ms and repeat!
26      }                                   // (Must not terminate)
27  }
28
29  int main(void) {
30      int id[4]={0,1,2,3};
31
32      // Perform any driver setup necessary...
33      colinit();                          // Initialize column toolbox
34      l88init();                          // Initialize 8*8 led toolbox
35
36      // Create (initially) needed application tasks...
37      xTaskCreate(ledPanel, "LED", 512, (void*)NULL, 2, NULL);
38      xTaskCreate(ledTask,
39      xTaskCreate(ledTask,
40      xTaskCreate(ledTask,
41
42      // Start the Scheduler!
43      vTaskStartScheduler();
44      // Should not reach here...
```

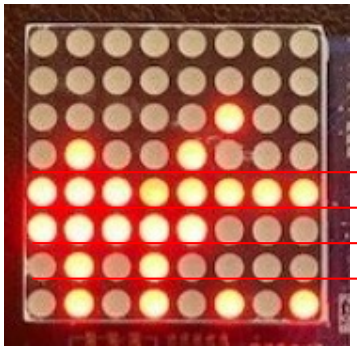
Visa “alltid” det 32-bits tal som xPortGetFreeHeapSize() ger!

Räkna upp “rätt” räknare, vänta “rätt” tid!

“TIPS”

Skapa tre trådar ur en “mall”, ”parameter” 0, 1 & 2.

xPortGetFreeHeapSize()



0x00000448

“ledTask1”
“ledTask2”
“ledTask3”

“Räknar upp” värdet i raden med 1 varje sekund, var annan sekund och var 3:e sekund

```
8   void ledPanel(void *pvParameters){      // The Led Panel Task:
9
10      while (1) {                         // Led Panel Super Loop...
11          l88row(colset());               // ...show next row...
12          vTaskDelay(1);                  // ...wait 2ms and repeat!
13
14          l88mem(0,
15          l88mem(1,
16          l88mem(2,
17          l88mem(3,
18      }                                   // (Must not terminate)
19  }
20
21  void ledTask(void *pvParameters){       // The Led Panel Task:
22                                          ters;
23                                          // Led Panel Super Loop...
24
25                                          // ...wait 2+id ms and repeat!
26      }                                   // (Must not terminate)
27  }
28
29  int main(void) {
30      int id[4]={0,1,2,3};
31
32      // Perform any driver setup necessary...
33
34
35
36      // Create (initially) needed application tasks...
37      xTaskCreate(ledPanel, "LED", 512, (void*)NULL, 2, NULL);
38      xTaskCreate(ledTask,
39      xTaskCreate(ledTask,
40      xTaskCreate(ledTask,
41
42      // Start the Scheduler!
43      vTaskStartScheduler();
44      // Should not reach here...
```

Visa “alltid” det 32-bits tal som xPortGetFreeHeapSize() ger!

A

Räkna upp “rätt” räknare, vänta “rätt” tid!

“TIPS”

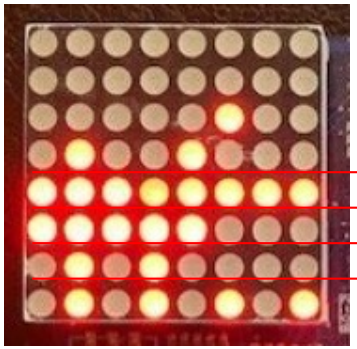
Grupparbete till x.45

Skapa tre trådar ur en “mall”, ”parameter” 0, 1 & 2.

B

LÖSNINGS FÖRSLAG

xPortGetFreeHeapSize()



0x00000448

“ledTask1”
“ledTask2”
“ledTask3”

“Räknar upp” värdet i raden med 1 varje sekund, var annan sekund och var 3:e sekund

```
8   void ledPanel(void *pvParameters){      // The Led Panel Task:
9       volatile char data[1800];
10      while (1) {                         // Led Panel Super Loop...
11          l88row(colset());               // ...show next row...
12          vTaskDelay(1);                  //    wait 2ms and repeat!
13
14
15
16
17
18
19  }
20
21  voi
22
23
24
25                                                              at!
26
27  }
28
29  int
30
31
32
33                                                              x
34                                                              ox
35
36
37
38
39
40
41
42
43      vTaskStartScheduler();
44      // Should not reach here...
```

Agenda

15.15 FreeRTOS

15.30 Grupparbete Tasks (Lab #1)

15.45 Apotek & Queue

Grupparbete Poisson process

Queue

16.15 Grupparbete Thread safe LCD

16.25 Lås & Semaforer

16.30 Grupparbete Simulering

16.45 Sammanfattning

Apotek

Apotek

(tråd)

```
while (1) {
    serveLCD();
    //serveCustomerRequests();
    serveClercRequests();
    serveSupervisorRequests();
}
```

(radio)

(tråd)

Manuellt +1

Manuellt -1

Vad händer om någon av funktionerna låser sig?

Apotek

(tråd)

```
while (1) {
xTaskCreate(lcdPanel, "LCD", 512
xTaskCreate(arrival, "ARR", 512,
xTaskCreate(clerk, "CL1", 512, (
xTaskCreate(clerk, "CL2", 512, (
}
```

(radio)

(tråd)

Manuellt +1

Manuellt -1

Bättre… **…för att kunna testa behöver vi dock kunder…**

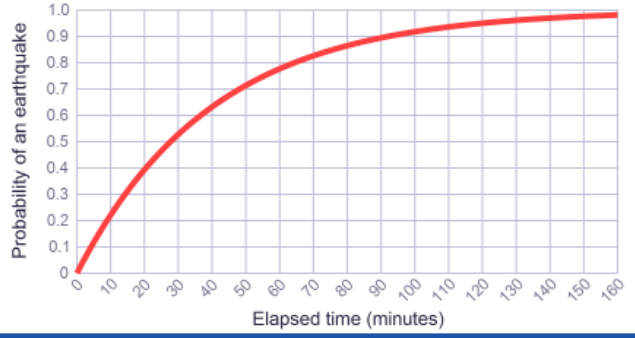
Kunder…

En ny kund anländer ca var 4:e minut…

M.a.o. kunder anländer med någon ”snittintensitet”…
…men tiden mellan två kunder är helt oberoende…
…den tiden är med andra ord inte normalfördelad…
…utan fördelad enligt en så kallad Poisson fördelning…
…all denna insikt utgör basen för så kallad kö-teori…
…som är en viktig del av modern programmeringsinsikt!

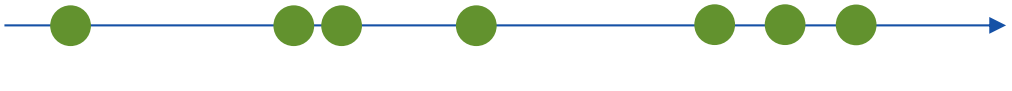
$\lambda = \frac{1}{40}$ and call it the *rate parameter*.

$$F(x) = 1 - e^{-\lambda x}$$



…och berättar i det här fallet att vi förväntar oss att något kommer att hända i snitt var fyrtionde minut!

Kunder…



En ny kund anländer ca var 4:e minut…

```
public final int poisson(double a) {
        double limit = Math.exp(-a), prod = nextDouble();
        int n;
        for (n = 0; prod >= limit; n++)
            prod *= nextDouble();
        return n;
        }
```

`nextDouble()` is a function from the `Random` package in Java that returns a uniformly distributed random `double`, for example `0.885598042879084`.

```
15    int main(void){
16        for (int i=0; i<10; i++) {
17            for (int j=0; j<10; j++)
18                printf("%2d, ",poisson(4));
19            printf("\n");
20        }
21    }
```

```
0,  4,  4,  2,  3,  2,  5,  6,  6,  5,
5,  2,  5,  5,  3,  5,  4,  8,  6,  4,
3,  8,  3,  1,  1,  4,  5,  6,  3,  5,
3,  5,  1,  5,  2,  2,  3,  7,  9,  9,
9,  1,  2,  8,  3,  5,  4,  2,  0,  6,
4,  4,  6,  4,  0,  4,  4,  3,  7,  2,
4,  4,  6,  3,  4,  4,  5,  7,  5,  3,
6,  4,  5,  5,  2,  4,  3,  4,  6,  0,
3,  1,  3,  4,  6,  2,  2,  5,  3,  5,
6,  4,  2,  2,  3,  5,  4,  7,  4,  3,
```

Lås och semaforer

En ny kund anländer ca var 4:e minut…

```
public final int poisson(double a) {
        double limit = Math.exp(-a), prod = nextDouble();
        int n;
        for (n = 0; prod >= limit; n++)
            prod *= nextDouble();
        return n;
        }
```

`nextDouble()` is a function from the `Random` package in Java that returns a uniformly distributed random `double`, for example `0.885598042879084`.

```
15    int main(void){
16        for (int i=0; i<10; i++) {
17            for (int j=0; j<10; j++)
18                printf("%2d, ",poisson(4));
19            printf("\n");
20        }
21    }
```

```
0,  4,  4,  2,  3,  2,  5,  6,  6,  5,
5,  2,  5,  5,  3,  5,  4,  8,  6,  4,
3,  8,  3,  1,  1,  4,  5,  6,  3,  5,
3,  5,  1,  5,  2,  2,  3,  7,  9,  9,
9,  1,  2,  8,  3,  5,  4,  2,  0,  6,
4,  4,  6,  4,  0,  4,  4,  3,  7,  2,
4,  4,  6,  3,  4,  4,  5,  7,  5,  3,
6,  4,  5,  5,  2,  4,  3,  4,  6,  0,
3,  1,  3,  4,  6,  2,  2,  5,  3,  5,
6,  4,  2,  2,  3,  5,  4,  7,  4,  3,
```

Kunder…

En ny kund anländer ca var 4:e minut…

```
public final int poisson(double a) {
        double limit = Math.exp(-a), prod = nextDouble();
        int n;
        for (n = 0; prod >= limit; n++)
            prod *= nextDouble();
        return n;
        }
```

`nextDouble()` is a function from the `Random` package in J
random `double`, for example `0.885598042879084`.

LÖSNINGS FÖRSLAG

```c
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include <limits.h>

int poisson(int lambda){
   double limit=exp(-lambda);
   double prod=(double)rand()/INT_MAX;
   int n;
   for (n=0; prod>=limit; n++)
      prod*=(double)rand()/INT_MAX;
   return n;
}

int main(void){
   for (int i=0; i<10; i++) {
      for (int j=0; j<10; j++)
         printf("%2d, ",poisson(4));
      printf("\n");
   }
}
```

Agenda

15.15 FreeRTOS

15.30 Grupparbete Tasks (Lab #1)

15.45 Apotek & Queue

Grupparbete Poisson process

Queue

16.15 Grupparbete Thread safe LCD

16.25 Lås & Semaforer

16.30 Grupparbete Simulering

16.45 Sammanfattning

Queue


```
xTaskCreate(lcdPanel, "LCD", 512
xTaskCreate(arrival, "ARR", 512,
xTaskCreate(clerk, "CL1", 512, (
xTaskCreate(clerk, "CL2", 512, (
```

…Hum, LCD-paketet från HE1028 är inte Thread Safe!

…varje “annan” tråds önskemål om “utskrifter” måste “köas upp” i sin helhet…

…det behövs någon form av buffert!

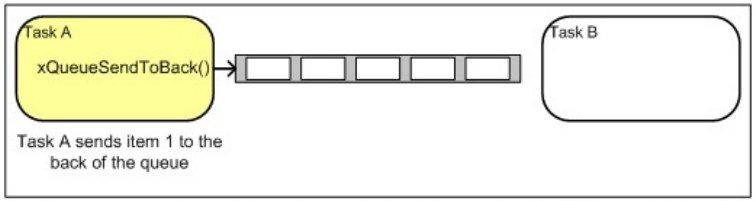
FreeRTOS Queues

[Inter-task communication and synchronisation]

FreeRTOS Queues

[See also Blocking on Multiple RTOS Objects]

Queues are the primary form of intertask communications. They can be used to send messages between tasks, and between interrupts and tasks. In most cases they are used as thread safe FIFO (First In First Out) buffers with new data being sent to the back of the queue, although data can also be sent to the front.



Writing to and reading from a queue. In this example the queue was created to hold 5 items, and the queue never becomes full.

Queue API functions permit a block time to be specified.

When a task attempts to read from an empty queue the task will be placed into the Blocked state (so it is not consuming any CPU time and other tasks can run) until either data becomes available on the queue, or the block time expires.

When a task attempts to write to a full queue the task will be placed into the Blocked state (so it is not consuming any CPU time and other tasks can run) until either space becomes available in the queue, or the block time expires.

If more than one task block on the same queue then the task with the highest priority will be the task that is unblocked first.

Queue

xQueueCreate

[Queue Management]

queue. h

```
QueueHandle_t xQueueCreate( UBaseType_t uxQueueLength,
                            UBaseType_t uxItemSize );
```

Creates a new queue and returns a handle by which the queue can be referenced. configSUPPORT_DYNAMIC_ALLOCATION must be set to 1 in FreeRTOSConfig.h, or left undefined (in which case it will default to 1), for this RTOS API function to be available.

Each queue requires RAM that is used to hold the queue state, and to hold the items that are contained in the queue (the queue storage area). If a queue is created using xQueueCreate() then the required RAM is automatically allocated from the FreeRTOS heap. If a queue is created using xQueueCreateStatic() then the RAM is provided by the application writer, which results in a greater number of parameters, but allows the RAM to be statically allocated at compile time. See the Static Vs Dynamic allocation page for more information.

Parameters:

uxQueueLength — The maximum number of items the queue can hold at any one time.

uxItemSize — The size, in bytes, required to hold each item in the queue.

Items are queued by copy, not by reference, so this is the number of bytes that will be copied for each queued item. Each item in the queue must be the same size.

Returns:

If the queue is created successfully then a handle to the created queue is returned. If the memory required to create the queue could not be allocated then NULL is returned.

Queue

xQueueSend

[Queue Management]

queue.h

```
BaseType_t xQueueSend(
                          QueueHandle_t xQueue,
                          const void * pvItemToQueue,
                          TickType_t xTicksToWait
                      );
```

This is a macro that calls xQueueGenericSend(). It is included for backward compatibility with versions of FreeRTOS that did not include the xQueueSendToFront() and xQueueSendToBack() macros. It is equivalent to xQueueSendToBack().

Post an item on a queue. The item is queued by copy, not by reference. This function must not be called from an interrupt service routine. See xQueueSendFromISR() for an alternative which may be used in an ISR.

Parameters:

xQueue	The handle to the queue on which the item is to be posted.
pvItemToQueue	A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.
xTicksToWait	The maximum amount of time the task should block waiting for space to become available on the queue, should it already be full. The call will return immediately if the queue is full and xTicksToWait is set to 0. The time is defined in tick periods so the constant portTICK_PERIOD_MS should be used to convert to real time if this is required. If INCLUDE_vTaskSuspend is set to '1' then specifying the block time as portMAX_DELAY will cause the task to block indefinitely (without a timeout).

Returns:pdTRUE if the item was successfully posted, otherwise errQUEUE_FULL. **Example usage:**

Queue

xQueueReceive

[Queue Management]

queue. h

```
BaseType_t xQueueReceive(
                            QueueHandle_t xQueue,
                            void *pvBuffer,
                            TickType_t xTicksToWait
                         );
```

This is a macro that calls the xQueueGenericReceive() function.

Receive an item from a queue. The item is received by copy so a buffer of adequate size must be provided. The number of bytes copied into the buffer was defined when the queue was created.

This function must not be used in an interrupt service routine. See xQueueReceiveFromISR for an alternative that can.

Parameters:

xQueue	The handle to the queue from which the item is to be received.
pvBuffer	Pointer to the buffer into which the received item will be copied.
xTicksToWait	The maximum amount of time the task should block waiting for an item to receive should the queue be empty at the time of the call. Setting xTicksToWait to 0 will cause the function to return immediately if the queue is empty. The time is defined in tick periods so the constant portTICK_PERIOD_MS should be used to convert to real time if this is required. If INCLUDE_vTaskSuspend is set to '1' then specifying the block time as portMAX_DELAY will cause the task to block indefinitely (without a timeout).

Returns:

pdTRUE if an item was successfully received from the queue, otherwise pdFALSE.

Vi gör LCD:n trådsäker med en kö!

Queue item
32bit word!

front
tail

xLCDQ

0x0000 0RNB
0x0000 0301

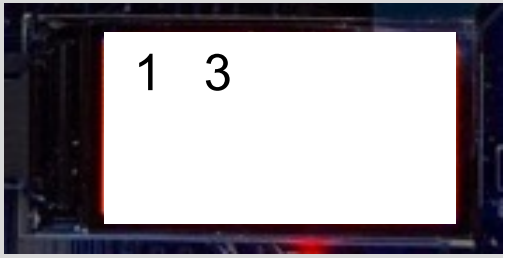
0x0000 CRNB
0x0000 CX00

Från arrival-tråden
Tid kvar till nästa kund
Ankomstnummer

LCD Thread

```
while (1) {
    vänta på uppdrag…
    …utför uppdrag!
}
```

"Könummer #1 kommer tas om 3 minuter(/sek)"



Active Object Pattern

```
15   QueueHandle_t xLCDQ;
119      xLCDQ      = xQueueCreate(10,sizeof(int));
```

Queue

Grupparbete


```
11  volatile int nnb=0;
15  QueueHandle_t xLCDQ;
```

```
19      xLCDQ      = xQueueCreate(10,sizeof(int));
```

```
71  void arrival(void *pvParameters){
72      int delta;
73      int msg;
74      while (1){
75          delta=poisson(4);
76          for (int i=0; i<delta; i++) {
77              msg=nnb+((delta-1-i)<<8);
78              vTaskDelay(500);
79          }
80
81
82          nnb++;
83
84      }
85  }
```

0x0000 0RNB

“Skriv” msg, max timeout

För simuleringen är 1 min = 1 sek!


```
40  void lcdPanel(void *pvParameters){      // The Led Panel Task:
41      int msg;
42      //Lcd_SetType(LCD_NORMAL);            // or use LCD_INVERTED!
43      Lcd_SetType(LCD_INVERTED);
44      Lcd_Init();
45      LCD_Clear(RED);
46      while (1) {                          // Lcd Panel Super Loop...
47          LCD_WR_Queue();
48          if (                                    )
49              switch (msg>>12) {
50                  case 0 : LCD_ShowNum(10,10,msg%256,3,WHITE);
51                           LCD_ShowNum(70,10,msg>>8,2,WHITE); break;
52                  case 1 : if (msg%256) {
53                               LCD_ShowNum(10,30,msg%256,5,WHITE);
54                               %16,2,WHITE);
55
56                               !", WHITE);
57
58                           break;
59
60                                                          );
61                                                          WHITE);
62
63                           LCD_ShowString(10,50, "IDLE!", WHITE);
64                  }
65                  break;
66          }
```

“Läs” msg, timeout=0

```
BaseType_t xQueueReceive(
                            QueueHandle_t xQueue,
                            void *pvBuffer,
                            TickType_t xTicksToWait
                          );
```

```
BaseType_t xQueueSend(
                            QueueHandle_t xQueue,
                            const void * pvItemToQueue,
                            TickType_t xTicksToWait
                          );
```

Hämta från Canvas main.c…. …fyll i de grå rutorna och provkör!

Queue


```
11  volati
15  QueueH
19      xL
71  void a
72      in
73      in
74      wh
75
76
77
78
79
80
81
82
83
84      }
85  }
```

```
ED!
p...
k;
E);
);
E);
);
```

Hämta från Canvas main.c…. …fyll i de grå rutorna och prövkör!

Agenda

15.15 FreeRTOS

15.30 Grupparbete Tasks (Lab #1)

15.45 Apotek & Queue

Grupparbete Poisson process

Queue

16.15 Grupparbete Thread safe LCD

16.25 Lås & Semaforer

16.30 Grupparbete Simulering

16.45 Sammanfattning

Lås och semaforer


```
11  volatile int nnb=0;
12  volatile int qnb=0;
13
14
15  QueueHandle_t xLCDQ;
```

```
71  void arrival(void *pvParameters){
72      int delta;
73      int msg;
74      while (1){
75          delta=poisson(4);
76          for (int i=0; i<delta; i++) {
77              msg=nnb+((delta-1-i)<<8);
78
79              vTaskDelay(500);
80          }
81
82          nnb++;
83          Det finns en ny kund!
84      }
85  }
```

```
117
118                                  main
```

Flera trådar använder variabeln qnb!

Hur vet klienterna att det finns en ny kund?

Lås och semaforer


```
11  volatile int nnb=0;
12  volatile int qnb=0;
13  SemaphoreHandle_t xQnbMutex;
14  SemaphoreHandle_t xCustWait;
15  QueueHandle_t xLCDQ;
```

```
71  void arrival(void *pvParameters){
72      int delta;
73      int msg;
74      while (1){
75          delta=poisson(4);
76          for (int i=0; i<delta; i++) {
77              msg=nnb+((delta-1-i)<<8);
78
79              vTaskDelay(500);
80          }
81
82          nnb++;
83          Det finns en ny kund!
84      }
85  }
```

```
117     xQnbMutex = xSemaphoreCreateMutex();
118     xCustWait = xSemaphoreCreateCounting(10, 0);
```

Flera trådar använder variabeln qnb!
Den behöver skyddas med ett lås!

Hur vet klienterna att det finns en ny kund? Det behövs en semafor!

Lås och semaforer

idl
0x0000 C000

Från clerk #c-tråden
N/A
Kön tom, vilar!

```
11  volatile int nnb=0;
12  volatile int qnb=0;
13  SemaphoreHandle_t xQnbMutex;
14  SemaphoreHandle_t xCustWait;
15  QueueHandle_t xLCDQ;
```

```
71  void arrival(void *pvParameters){
72      int delta;
73      int msg;
74      while (1){
75          delta=poisson(4);
76          for (int i=0; i<delta; i++) {
77              msg=nnb+((delta-1-i)<<8);
78
79              vTaskDelay(500);
80          }
81
82          nnb++;
83          Det finns en ny kund!
84      }
85  }
```

```
117     xQnbMutex = xSemaphoreCreateMutex();
118     xCustWait = xSemaphoreCreateCounting(10, 0);
```

```
87  void clerk(void *pvParameters){
88      int cid=(*(int *)pvParameters)<<12;
89      int cst;
90      int cnb;
91      int msg;
92      int idl=(*(int *)pvParameters)<<12;
93      while (1){
94          if ( Ingen kö? )
95              xQueueSend(xLCDQ, &idl, portMAX_DELAY);
96              Vänta på att det finns en ny kund…
97
98                  cnb=++qnb;
99
100         cst=poisson(6);
101         for (int i=0; i<cst; i++) {
102             msg=cid+((cst-1-i)<<8)+cnb;
103             xQueueSend(xLCDQ, &msg, portMAX_DELAY);
104             vTaskDelay(500);
105         }
106         vTaskDelay(500);
107     }
108 }
```

*2

NY TASK: clerk (2 stycken!)

```
125     xTaskCreate(clerk, "CL1", 512, (void*)&id[1], 2, NULL);
126     xTaskCreate(clerk, "CL2", 512, (void*)&id[2], 2, NULL);
```

Lås och semaforer


```
11  volatile int nnb=0;
12  volatile int qnb=0;
13  SemaphoreHandle_t xQnbMutex;
14  SemaphoreHandle_t xCustWait;
15  QueueHandle_t xLCDQ;
```

```
71  void arrival(void *pvParameters){
72      int delta;
73      int msg;
74      while (1){
75          delta=poisson(4);
76          for (int i=0; i<delta; i++) {
77              msg=nnb+((delta-1-i)<<8);
78
79              vTaskDelay(500);
80          }
81
82          nnb++;
83          xSemaphoreGive(xCustWait);
84      }
85  }
```

Det finns en ny kund!

```
117     xQnbMutex = xSemaphoreCreateMutex();
118     xCustWait = xSemaphoreCreateCounting(10, 0);
```

```
87  void clerk(void *pvParameters){
88      int cid=(*(int *)pvParameters)<<12;
89      int cst;
90      int cnb;
91      int msg;
92      int idl=(*(int *)pvParameters)<<12;
93      while (1){
94          if (uxQueueMessagesWaiting(xLCDQ)==0)
95              xQueueSend(xLCDQ, &idl, portMAX_DELAY);
96          xSemaphoreTake(xCustWait, portMAX_DELAY);
97
98              cnb=++qnb;
99
100         cst=poisson(6);
101         for (int i=0; i<cst; i++) {
102             msg=cid+((cst-1-i)<<8)+cnb;
103             xQueueSend(xLCDQ, &msg, portMAX_DELAY);
104             vTaskDelay(500);
105         }
106         vTaskDelay(500);
107     }
108 }
```

Ingen kö?

Vänta på att det finns en ny kund…

*2

```
125     xTaskCreate(clerk, "CL1", 512, (void*)&id[1], 2, NULL);
126     xTaskCreate(clerk, "CL2", 512, (void*)&id[2], 2, NULL);
```

Lås och semaforer

0x0000 CRNB ← Från clerk #c-tråden
← Tid kvar att betjäna kund
← Betjänar kund #NB,
om 0 = sover, ingen kö!

```
11  volatile int nnb=0;
12  volatile int qnb=0;
13  SemaphoreHandle_t xQnbMutex;
14  SemaphoreHandle_t xCustWait;
15  QueueHandle_t xLCDQ;
```


```
71  void arrival(void *pvParameters){
72      int delta;
73      int msg;
74      while (1){
75          delta=poisson(4);
76          for (int i=0; i<delta; i++) {
77              msg=nnb+((delta-1-i)<<8);
78
79              vTaskDelay(500);
80          }
81
82          nnb++;
83          Det finns en ny kund!
84      }
85  }
```

```
117     xQnbMutex = xSemaphoreCreateMutex();
118     xCustWait = xSemaphoreCreateCounting(10, 0);
```

```
87  void clerk(void *pvParameters){
88      int cid=(*(int *)pvParameters)<<12;
89      int cst;
90      int cnb;
91      int msg;
92      int idl=(*(int *)pvParameters)<<12;
93      while (1){
94          if (    Ingen kö?    )
95              xQueueSend(xLCDQ, &idl, portMAX_DELAY);
96          Vänta på att det finns en ny kund…
97          "Ta" låset
98              cnb=++qnb;
99          "Lämna tillbaks" låset
100         cst=poisson(6);
101         for (int i=0; i<cst; i++) {
102             msg=cid+((cst-1-i)<<8)+cnb;
103             xQueueSend(xLCDQ, &msg, portMAX_DELAY);
104             vTaskDelay(500);
105         }
106         vTaskDelay(500);
107     }
108 }
```

*2

```
125     xTaskCreate(clerk, "CL1", 512, (void*)&id[1], 2, NULL);
126     xTaskCreate(clerk, "CL2", 512, (void*)&id[2], 2, NULL);
```

Lås och semaforer

- Binary Semaphores
- Counting Semaphores
- **Mutexes**
- Recursive Mutexes
- ⊞ Direct To Task Notifications
- ⊞ Stream & Message Buffers
- ⊞ Software Timers
- Event Groups (or "Flags")
- Source Code Organization
- FreeRTOSConfig.h
- Static Vs Dynamic Memory
- Heap Memory Management
- Stack Overflow Protection
- Creating a New Project

FreeRTOS Mutexes

Mutexes are binary semaphores that include a priority inheritance mechanism. Whereas binary semaphores are the better choice for implementing synchronisation (between tasks or between tasks and an interrupt), mutexes are the better choice for implementing simple mutual exclusion (hence 'MUT'ual 'EX'clusion).

When used for mutual exclusion the mutex acts like a token that is used to guard a resource. When a task wishes to access the resource it must first obtain ('take') the token. When it has finished with the resource it must 'give' the token back - allowing other tasks the opportunity to access the same resource.

Mutexes use the same semaphore access API functions so also permit a block time to be specified. The block time indicates the maximum number of 'ticks' that a task should enter the Blocked state when attempting to 'take' a mutex if the mutex is not immediately available. Unlike binary semaphores however - mutexes employ priority inheritance. This means that if a high priority task blocks while attempting to obtain a mutex (token) that is currently held by a lower priority task, then the priority of the task holding the token is temporarily raised to that of the blocking task. This mechanism is designed to ensure the higher priority task is kept in the blocked state for the shortest time possible, and in so doing minimise the 'priority inversion' that has already occurred.

Priority inheritance does not cure priority inversion! It just minimises its effect in some situations. Hard real time applications should be designed such that priority inversion does not happen in the first place.

semphr. h (Låset är inte taget, när det har skapats.)

```
SemaphoreHandle_t xSemaphoreCreateMutex( void )

xSemaphoreTake( SemaphoreHandle_t xSemaphore,
                TickType_t xTicksToWait );

xSemaphoreGive( SemaphoreHandle_t xSemaphore );
```

Lås och semaforer

FreeRTOS Counting Semaphores

TIP: 'Task Notifications' can provide a light weight alternative to counting semaphores in many situations

Just as binary semaphores can be thought of as queues of length one, counting semaphores can be thought of as queues of length greater than one. Again, users of the semaphore are not interested in the data that is stored in the queue - just whether or not the queue is empty or not.

Counting semaphores are typically used for two things:

1. Counting events.

   In this usage scenario an event handler will 'give' a semaphore each time an event occurs (incrementing the semaphore count value), and a handler task will 'take' a semaphore each time it processes an event (decrementing the semaphore count value). The count value is therefore the difference between the number of events that have occurred and the number that have been processed. In this case it is desirable for the count value to be zero when the semaphore is created.

```
SemaphoreHandle_t xSemaphoreCreateCounting( UBaseType_t uxMaxCount,
                                            UBaseType_t uxInitialCount);

xSemaphoreGive( SemaphoreHandle_t xSemaphore );

xSemaphoreTake( SemaphoreHandle_t xSemaphore,
                TickType_t xTicksToWait );

UBaseType_t uxSemaphoreGetCount( SemaphoreHandle_t xSemaphore );
```

Lås och semaforer

```
xSemaphoreGive( SemaphoreHandle_t xSemaphore );

xSemaphoreTake( SemaphoreHandle_t xSemaphore,
                TickType_t xTicksToWait );

UBaseType_t uxSemaphoreGetCount( SemaphoreHandle_t xSemaphore );
```

```
11  volatile int nnb=0;
12  volatile int qnb=0;
13  SemaphoreHandle_t xQnbMutex;
14  SemaphoreHandle_t xCustWait;
15  QueueHandle_t xLCDQ;
```


```
71  void arrival(void *pvParameters){
72      int delta;
73      int msg;
74      while (1){
75          delta=poisson(4);
76          for (int i=0; i<delta; i++) {
77              msg=nnb+((delta-1-i)<<8);
78
79              vTaskDelay(500);
80          }
81
82          nnb++;
83
84      }
85  }
```

Det finns en ny kund!

```
87  void clerk(void *pvParameters){
88      int cid=(*(int *)pvParameters)<<12;
89      int cst;
90      int cnb;
91      int msg;
92      int idl=(*(int *)pvParameters)<<12;
93      while (1){
94          if (
95            xQueueSend(xLCDQ, &idl, portMAX_DELAY);
96
97
98                cnb=++qnb;
99
100         cst=poisson(6);
101         for (int i=0; i<cst; i++) {
102             msg=cid+((cst-1-i)<<8)+cnb;
103             xQueueSend(xLCDQ, &msg, portMAX_DELAY);
104             vTaskDelay(500);
105         }
106         vTaskDelay(500);
107     }
108 }
```

Ingen kö?

Vänta på att det finns en ny kund…
"Ta" låset

"Lämna tillbaks" låset

*2

```
117     xQnbMutex = xSemaphoreCreateMutex();
118     xCustWait = xSemaphoreCreateCounting(10, 0);
```

Glöm inte

GRUPPARBETE: Hämta, och kopiera in clerk från Canvas!

```
125     xTaskCreate(clerk, "CL1", 512, (void*)&id[1], 2, NULL);
126     xTaskCreate(clerk, "CL2", 512, (void*)&id[2], 2, NULL);
```

Lås och semaforer

LÖSNINGS FÖRSLAG

```
11
12
13
14
15
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
117
118
```

```
125     xTaskCreate(clerk, "CL1", 512, (void*)&id[1], 2, NULL);
126     xTaskCreate(clerk, "CL2", 512, (void*)&id[2], 2, NULL);
```

Begrepp

Task
Poisson
Queue
Shared data
Critical sector
Mutual exclusion
Lock
Semaphore